Things learned in our data engineering journey and ideas on data and engineering.

Follow publication

A non-beginner Data Engineering Roadmap — 2025 Edition

Want to become a better Data Engineer this year? Here's my non-beginner Roadmap

Ernani Castro · Follow

Published in Data Engineer Things · 14 min read · Feb 17, 2025

1.3K 10

Me after years using python.

Before starting this post, I want to acknowledge that soft and hard skills are equally important. Data people exist to deliver business value, or more broadly read facts from a pool of ever-growing data.

But, even with a bunch of posts talking about soft skills, at the end of the day, we're being paid for the technical skills we have, and the ability we have to deliver high-quality code within tight deadlines, or even if you're Senior+ and spend less and less time coding, you're still demanded to look through a crazy landscape of thousands of tools, providers hiking their prices every year (some lowering their prices), stakeholders asking to throw every single GenAI product at the wall and seeing what sticks, and everyone worried about the future of the job market with a possible AGI coming even though we all know that Transformers can't become AGI without technological breakthroughs that didn't happen yet.

Ok, we can **breathe** again.

> In this post I'll guide you through the consolidated technologies used by data engineers beyond the starter stack.

. . .

If you're reading this, I expect you to have these skills already:

Python / SQL / Data Modeling / Spark (or any flavor of RDD) / Git /

Any cloud analytics stack

I'll focus on established and mainstream tools, setting aside emerging favorites in the data community such as DuckDB, Polars, and uv.

. . .

We'll cover these topics:

- Books
- Leetcode's SQL 50
- Open Table Format
- Data Lake + Data Warehouse
- Modern OLAP
- Streaming
- Real-time Processing
- Other Flavors of DB
- Orchestration
- Data Quality
- CI/CD
- AI Code Editor
- Visualization
- Data Validation
- Dependency Management and Packaging
- Containers
- Infrastructure as Code (IaC)
- Cloud Security & Networking
- Web Framework
- API Development
- OAuth2.0 and other kinds of auth
- Certifications
- Conferences
- Project Ideas

⚠️ This post is not data-driven, so read it cautiously and apply it to the context you or your employer are currently in.

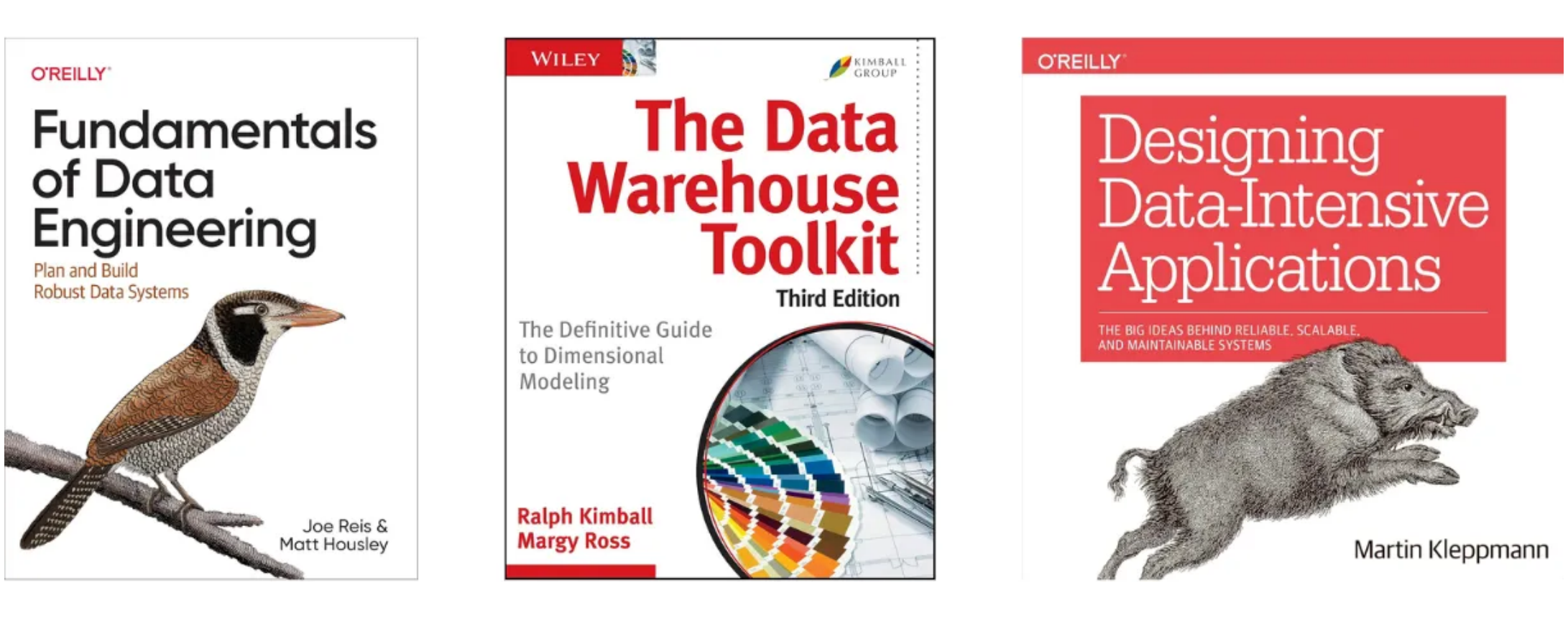
We'll not cover Kubernetes.

Yes, it's a lot, so **let's get started:**

Books

Books are at the forefront of this post because I'm not aiming to reiterate the established knowledge in the same books I've studied.

Therefore, if you haven't read them yet or aren't fully at ease with their content, please bookmark this post and return once you've gone through those texts.



O'REILLY
Fundamentals of Data Engineering
Plan and Build Robust Data Systems
Joe Reis & Matt Housley
WILEY
The Data Warehouse Toolkit
Third Edition
The Definitive Guide to Dimensional Modeling
Ralph Kimball
Margy Ross
O'REILLY
Designing Data-Intensive Applications
THE BIG IDEAS BEHIND RELIABLE, SCALABLE, AND MAINTAINABLE SYSTEMS
Martin Kleppmann

Fundamentals of Data Engineering

Joe Reis and Matt Housley bring future proof of what it means to be a data engineer and go through the concepts of data generation, ingestion, orchestration, transformation, storage, and governance.

The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling

So classic people refer to it as just Kimball, we have all been there, if you haven't read it already or understand what is the Kimball Model, time to go for it.

The book introduces us to the concepts of

- Star Schema Design
- Dimension and Fact Table Architecture
- Slowly Changing Dimensions (SCDs)
- Conformed Dimensions
- Bus Architecture

Designing Data-Intensive Applications

This book is important for every person doing software, it will give you insights on how to decide which technologies to use and how they work under the hood.

The book combines real-world examples with the research behind databases and architectures of modern data stacks teaching how to navigate the decisions and trade-offs.

. . .

Leetcode's SQL 50

While some suggest completing three problems daily, we'll take a more sustainable approach throughout the year.

This year, we'll solve problems at your own pace, focusing on understanding how to optimize time and space complexity rather than rushing through the exercises.

The graph below displays the distribution of successful submission times (solutions that passed all test cases).

Learning goals:

Our objective is to optimize our solution to align with the most efficient group of users, represented by the leftmost peak of the distribution curve. While multiple peaks may exist, focusing on the leftmost one represents the optimal balance between correctness and execution time

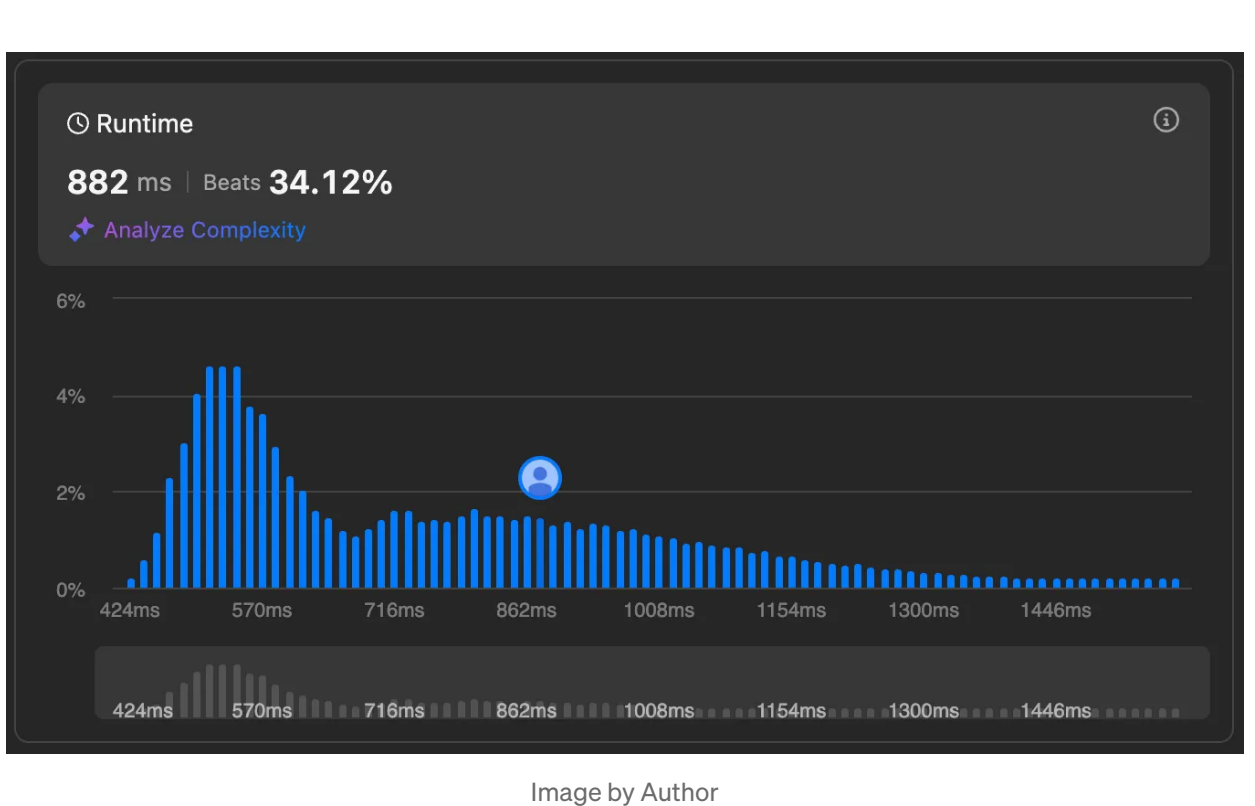


Image by Author

Resources:

Leetcode SQL 50

Hacker Rank SQL

. . .

Open Table Format

In 2024, Apache Iceberg emerged as the clear frontrunner in the data lake table format battle, particularly after AWS threw its weight behind it with native S3 Tables integration, effectively overshadowing Delta Lake.

If you're beyond the basics and still working directly with raw Parquet files (presumably with gzip compression), it's time to level up your data architecture game by mastering Iceberg's powerful features.

Learning goals:

- Understand use cases in which your team will benefit from using Open Tables
- Hidden Partitioning
- Schema Evolution
- Time Travel
- Data Compaction

Resources:

Tabular, the creators of Iceberg, acquired by Databricks have their Introduction Guide.

Dremio has a perfect YouTube Playlist for more visual learners, like me.

. . .

Data Lake + Data Warehouse

By now, you should be very familiar with current concepts like Data Lake, Data Warehouse, LakeHouse, Mart, etc

In the last decade or so, Storage become way cheaper than Compute, so the concept of Lake House was born.

But we, as Data Engineers, need to understand why usually companies decide between Snowflake vs Databricks as they were the same technology by two different vendors.

Our goal here is to understand why and when to choose a Data Lake solution, and when to select a Data Warehouse.

But if you don't, here's a video by Databricks presenting the concepts:

Intro to Data Lakehouse
databricks
What is a data lakehouse?
History of data management
Ansehen auf YouTube

Learning goals:

- Differences between Data Lake and Data Warehouse
- Snowflake vs Databricks

Data Lake Alternatives:

- Tabular
- Microsoft
- Databricks
- Delta Lake

Data Warehouse Alternatives:

- Snowflake
- Redshift
- BigQuery

. . .

Modern OLAP

The explosive momentum of OLAP (*Online Analytical Processing)* databases like DuckDB, Druid, Pinot, and Clickhouse demands attention. Despite any initial reservations, their growing adoption makes understanding these analytical processing systems' fundamentals a worthwhile investment for data professionals.

Learning goals:

- Learn when should we consider using an OLAP
- Understand why there are so many newcomers, what do Microsoft and Oracle miss here?

Resources:

ClickHouse Basic Tutorial

Timescale: Understanding OLAP

Alternatives:

- Apache Druid
- ClickHouse
- Apache Pinot
- Apache Kylin
- DuckDB

Streaming

We'll leverage Confluent's 30-day free trial to learn proper streaming implementation.

While Apache Kafka is widely adopted, you're free to explore other streaming platforms that best suit your needs.

Our goal here is to understand why most companies are adopting Apache Kafka as their main Data Source for both streaming and batch sinking.

Learning goals:

- Understand the use cases where Streaming is the best option
- Kappa vs Lambda

Resources:

Fundamentals Workshop: Apache Kafka 101

Real-time Processing

Along with Streaming, we need to introduce the concept of Real-Time Processing, which is usually implemented along with Apache Flink or Spark Streaming.

Learning goals:

- Understand the common Use Cases for Real-Time i.e. data analytics, fraud detection, rule-based alerting, etc.

Resources:

Timescale: Data Analytics vs. Real-Time Analytics: How to Pick Your Database

Confluent: Apache Flink — A Complete Introduction

Other Flavors of DB

Key-Value / In-Memory / Document / Wide-Column / Graph / Time-Series

Yes, there are a lot of Database types, and Postgres always seems the best one to go.

But we need to be able to understand as consumers or producers, the other options.

Learning goals:

- Get an overview of the Pros and Cons for each Database Type, including the ability and interest of your team to learn a new concept
- Acquire a wider range of options than just Postgres and MongoDB

Resources:

Choosing an AWS Database Service

Orchestration

While there are various orchestration tools available, including Dagster, Apache Airflow remains the industry standard for managing data pipelines at scale, despite its known limitations.

Although many practitioners have concerns about Airflow's current version, and there's anticipation for version 3.0's improvements, it continues to be the most robust and widely adopted solution.

Learning goals:

The objective here is to explore orchestration's full potential beyond simple task scheduling, leveraging its rich ecosystem of integrations to enhance workflow efficiency.

For beginners, start with the powerful combination of Apache Airflow and dbt Core, which provides a solid foundation for data pipeline orchestration.

Alternatives:

- Mage
- Astronomer
- Prefect
- Dagster

Resources:

Airflow Documentation

The Complete Hands-On Introduction to Apache Airflow (Paid) on Udemy by Marc Lambert (you should also follow him!)

Data Quality

Bringing modularity and simplicity to the transformation layer of modern ELT pipelines, enabling data teams to build and manage data transformations using software engineering best practices.

As the industry shifts from ETL to ELT workflows, dbt Core offers a free, open-source solution for handling the critical transformation phase with version control, testing, and documentation capabilities.

Learning goals:

- Determine the appropriate level of Data Quality checks by analyzing similar use cases you aim to implement.

Resources:

dbt Fundamentals, from dbt Labs.

Alternatives:

- dbt
- Great Expectations
- Coalesce
- Soda

CI/CD

It used to be harder to set up CI/CD, but now we have GitHub Actions and other options that integrate with our Git workflow.

So it's time to learn the basics of GH Actions.

```
name: GitHub Actions Demo
run-name: ${{ github.actor }} is testing out GitHub Actions 🚀
on: [push]
jobs:
  Explore-GitHub-Actions:
    runs-on: ubuntu-latest
    steps:
      - run: echo "🎉 The job was automatically triggered by a ${{ github.event_
      - run: echo "🐧 This job is now running on a ${{ runner.os }} server hoste
      - run: echo "🔎 The name of your branch is ${{ github.ref }} and your repo
      - name: Check out repository code
        uses: actions/checkout@v4
      - run: echo "💡 The ${{ github.repository }} repository has been cloned to
      - run: echo "🖥️ The workflow is now ready to test your code on the runner.
      - name: List files in the repository
        run: |
          ls ${{ github.workspace }}
      - run: echo "🍏 This job's status is ${{ job.status }}."
```

Learning goals:

- Learn common use cases for CI/CD to save your time, not only for build and deploy

Resources:

GH Actions: Use Cases and Examples

Awesome Actions

AI Code Editor

It's being reported that Engineers' productivity went up by 25% when using AI-aided coding, both GH Copilot and Cursor make it easy to integrate with our coding workflow instead of making it a cumbersome copy-paste activity.

Learning goals:

- Test a couple of AI Code editors and see which one adapts well into your current workflow

Resources:

Github Copilot Free

Cursor Features

Visualization

Data visualization extends beyond the traditional Data Analyst domain. Understanding how end users interact with data — whether through direct exports or visualization tools — is crucial for effective data engineering.

While tools like Microsoft Power Query can handle basic data transformations, they're unsuitable for large-scale data processing. These desktop-based solutions often fail when dealing with datasets that exceed local memory constraints, making them an inappropriate choice for enterprise-level data pipelines.

Learning goals:

- Understand the limitations of the most used DataViz tools
- Know common patterns that should be replaced with ETL

Resources:

Datacamp: Power BI vs. Tableau

23 Best DataViz Tools

Alternatives:

- Tableau
- Power BI
- Streamlit
- Preset
- Metabase
- Qlik
- Klipfolio

Data Validation

Pydantic has established itself as a game-changing data validation library in Python's ecosystem, transforming how we handle data structures and type checking.

By supercharging Python's native dataclasses with intuitive validation capabilities, Pydantic has accomplished what the standard library's dataclasses couldn't: widespread adoption in the data community. Its seamless integration with FastAPI further cements its position as an essential tool for modern data engineering.

The library's elegant approach to runtime type checking and data validation has made previously tedious input validation tasks both robust and maintainable.

Here's a simple example of how Pydantic works, from Pydantic Documentation (We'll validate Json data):

```
[
  {
      "name": "John Doe",
      "age": 30,
      "email": "john@example.com"
  },
  {
      "age": -30,
      "email": "not-an-email-address"
  }
]
```

In the snippet above we have a List of the Class Person, and while the first element of the List passes our validation, on the second element, age is negative and email doesn't pass Pydantic native validation.

```
import pathlib
from typing import List

from pydantic import BaseModel, EmailStr, PositiveInt, TypeAdapter

class Person(BaseModel):
    name: str
    age: PositiveInt
    email: EmailStr

person_list_adapter = TypeAdapter(List[Person])
json_string = pathlib.Path('people.json').read_text()
people = person_list_adapter.validate_json(json_string)
print(people)
#> [Person(name='John Doe', age=30, email='john@example.com'), Person(name='Jane
```

So after running this code, Pydantic will raise a ValidationError.

Learning goals:

- Understand the difference between Dataclasses/Pydantic and regular Data Quality checks.

Resources:

Pydantic Docs

Dependency Management and Packaging

Dependency management remains one of the most frustrating challenges in data engineering.

The notorious "it works on my machine" syndrome often strikes when attempting to revive a project after six months or when team members try to run your code in their local environment.

What seemed like a perfectly functional application can quickly turn into a maze of conflicting package versions and incompatible dependencies.

Here's an example from Poetry documentation of how poetry uses pyproject.toml to generate a lock file.

```
[project]
name = "poetry-demo"
version = "0.1.0"
description = ""
authors = [
    {name = "Sébastien Eustace", email = "sebastien@eustace.io"}
]
readme = "README.md"
requires-python = ">=3.9"
dependencies = [
]

[build-system]
requires = ["poetry-core>=2.0.0,<3.0.0"]
build-backend = "poetry.core.masonry.api"
```

and an example of how a poetry.lock would look like:

```
[[package]]
name = "flask"
version = "1.1.2"
description = "A simple framework for building complex web applications."
category = "main"
optional = false
python-versions = ">=2.7, !=3.0.*, !=3.1.*, !=3.2.*, !=3.3.*, !=3.4.*, <4"

[package.dependencies]
click = ">=5.1"
itsdangerous = ">=0.24"
jinja2 = ">=2.10.1"
werkzeug = ">=0.15"
[[package]]
name = "requests"
version = "2.1.4"
description = "Python HTTP for Humans"
files = [
    {file = "requests-2.1.4-py2.py3-none-any.whl", hash = "sha256:1234..."},
    {file = "requests-2.1.4.tar.gz", hash = "sha256:5678..."}
]
[metadata]
lock-version = "2.0"
python-versions = "^3.12"
content-hash = "sha256:9abc..."
```

Learning goals:

- Understand why Dependency management is important for the Future You and the love of your teammates.

Resources:

Poetry Docs

Containers

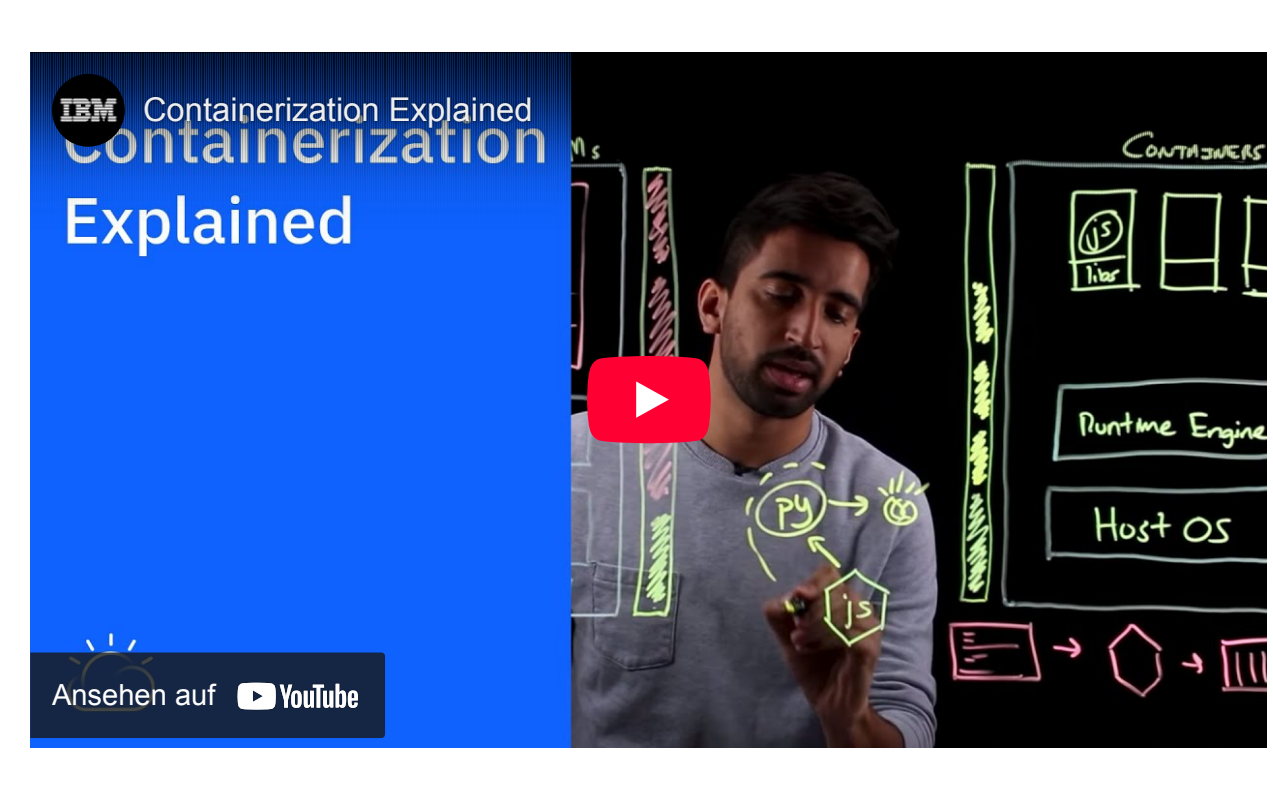
Beyond building and running containers, crafting efficient Dockerfiles is crucial.

As Data Engineers, we'll prioritize two key learning aspects: properly exposing database ports and optimizing image size for maximum performance

Learning goals:

- Understanding Containerization
- Dockerfile: A text file containing instructions to build a Docker image
- Port Mapping: Connect container ports to host ports (-p 80:80)
- Volumes: Persistent data storage outside containers
- Networks: Connect containers to communicate
- Environment Variables: Configure containers at runtime
- Docker Compose: Tool for defining multi-container applications

Resources:



Docker Fundamentals

Learn just enough docker to to do simple things

learn.cantrill.io

Docker Fundamentals

Docker 101

Infrastructure as Code (IaC)

Infrastructure provisioning through manual clicking in cloud consoles is strongly discouraged at this stage.

Instead, adopt Infrastructure as Code (IaC) practices with Terraform to manage your infrastructure.

You can start with existing modules from the HashiCorp Registry rather than creating custom ones, making the learning curve more manageable.

While writing Terraform code itself is straightforward, mastering the underlying cloud infrastructure concepts can be challenging. Rather than diving into complex resources, let's begin with something practical yet fundamental: creating a fully configured storage solution. This is an ideal starting point since storage services often come with free tiers, allowing you to experiment and learn without incurring costs.

Example of main.tf to create an EC2 instance (AWS basic compute) from an Image:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 4.16"
    }
  }

  required_version = ">= 1.2.0"
}

provider "aws" {
  region  = "us-west-2"
}

resource "aws_instance" "app_server" {
  ami           = "ami-830c94e3"
  instance_type = "t2.micro"

  tags = {
    Name = "ExampleAppServerInstance"
  }
}
```

Learning goals:

- Understanding Infrastructure as Code (IaC) principles
- Knowing the Terraform workflow (init, plan, apply, destroy)
- Define input variables
- Manage and use Outputs
- Managing state files and understanding their importance

Resources:

Terraform official guide

Cloud Security & Networking

We often start with just enough setup to get our cloud environments running.

However, it's now time to implement the principle of least privilege and learn to operate within private subnets, i.e., subnets that do not allow direct internet access.

> *Friends don't let friends use IAM Users*

Learning goals:

- Learn how to protect the data you're working on, removing internet connection and granting only necessary access.

Resources:

Security best practices in IAM

Security best practices for your VPC

OWASP Top Ten

Web Framework

Having an API to fetch data is much better than dealing with ever-changing web pages or Excel files, right?

We'll follow the official setup guide to create a local API, laying the groundwork for more advanced applications ahead.

How easy it is to run FastAPI?

1. Paste this code.

```
from fastapi import FastAPI
app = FastAPI()

@app.get("/")
async def root():
    return {"message": "Hello World"}
```

2. Run (not literally).

```
fastapi dev main.py
```

Make sure to follow Tiangolo and Marcelo Tryle to keep updated on the latest releases.

Learning goals:

- Learn enough to make your data available as an API without improvising with Requests lib.
- Understand how to work with Async functions.
- Understand what role Pydantic plays in FastAPI.

Resources:

FastAPI — Learn

API Development

An API management tool remains essential in a Data Engineer's toolkit.

While Postman's free tier has become increasingly limited compared to its early days, mastering an API testing platform is non-negotiable in our field.

Whether for validating endpoints before pipeline integration or importing OpenAPI (formerly Swagger) specifications, having a reliable API testing environment streamlines the development process and reduces integration headaches downstream.

Learning goals:

- We aim to set up a development environment, import an Open API schema, or build our own Open API Schema.

Resources:

How to Define an Open API Schema

Alternatives:

OAuth2.0 and other kinds of auth

Learning and doing beginner projects it's common to use Public APIs to get data, but in a working setting, that's not the case.

You'll probably need to use a form of authentication, OAuth2.0 being the most common. And while putting client_id/client_secret in Postman or your favorite BI tool, as a Data Engineer we'll need to add this to our Python Code base.

Basic Authentication Types

No Auth / API Key / Bearer Token

Advanced Authentication Types

JWT Bearer / Basic Auth / OAuth 2.0

In the image below is the classic Spotify API, and while getting the refresh token is an easy task, getting the proper setup for the user to accept the authorization scope, is not the easiest task (as we're not full stack engineers)

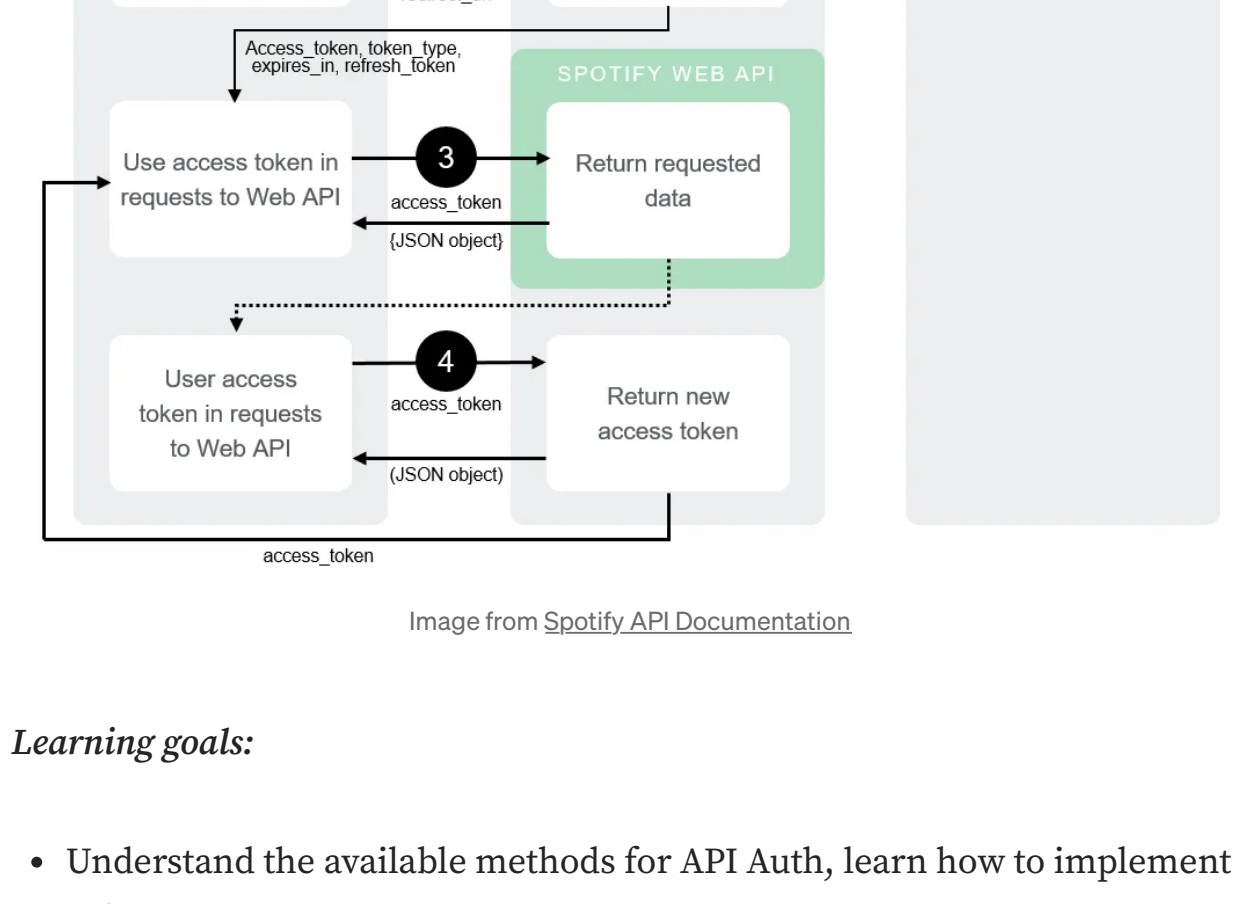

APPLICATION
SPOTIFY ACCOUNTS SERVICE
USER
Request authorization to access data
1
client_id, response_type, redirect_uri, state, scope
Display scopes and prompt user to login (if required)
Log in, authorize access
code, state
Request access and refresh tokens
2
client_id, client_secret, grant_type, code, redirect_uri
Return access and refresh tokens
Access_token, token_type, expires_in, refresh_token
SPOTIFY WEB API
Use access token in requests to Web API
3
access_token
{JSON object}
Return requested data
User access token in requests to Web API
4
access_token
{JSON object}
Return new access token
access_token

Image from Spotify API Documentation

Learning goals:

- Understand the available methods for API Auth, learn how to implement it fast, and debug it.

Resources:

API authentication and authorization in Postman

Kong's Insomnia: Authentication

Certifications

Set aside the never-ending debate between hands-on projects and certifications.

Now is the time to pursue an Associate-Level Certification with the cloud provider you previously selected.

I recommend a more general credential like the AWS Solutions Architect Associate, but the choice is yours — even if you're focusing on data engineering.

Expect to spend around two months preparing for the exam.

Resources:

AWS Certified Data Engineer — Associate

Microsoft Certified: Azure Data Engineer Associate

GCP Professional Data Engineer

Conferences

Usually, company conferences will be when they do the most simultaneous releases, we need to check the top announcements for our favorite vendor to stay on the top of our game.

- Data + AI Summit — June 9–12, San Francisco, CA, USA + Online

- Snowflake Summit 2025, will take place from June 2–5, 2025, at the Moscone Center in San Francisco
- Google Cloud Next 2025, will be held from April 9–11, 2025, at the Mandalay Bay Convention Center in Las Vegas
- Coalesce 2025, by dbt Labs, will take place in Las Vegas from October 4–11, 2025
- Microsoft Ignite 2025: will be held November 19–21, 2025
- AWS re:Invent 2025 will take place December 1–5, 2025, in Las Vegas, Nevada

Project Ideas

As a great contemporary philosopher said (I'll give the credit if he ends up reading this article):

> "Studying without practicing is entertainment"

Learning goals:

- Getting Hands-on experience on some of the topics from this article
- Building a GitHub Portfolio

Resources:

Datacamp: Top 11 Data Engineering Projects

. . .

> 💡 *Did you know that you can "Clap" up to 50 times in Medium?*

When reading an article on Medium, click the clap icon (👏) and hold it down or click repeatedly to give anywhere from 1 to 50 claps.

Loved this article ❤️? Share it on Linkedin and tag me!

Data Engineering Analytics Software Engineering Data Analytics Editors Pick

1.3K 10

Published in Data Engineer Things

22K Followers · Last published 6 days ago

Things learned in our data engineering journey and ideas on data and engineering.

Follow

Written by Ernani Castro

181 Followers · 158 Following

Sharing my views on how to become a generalist in the Data space. Senior DE | AWS SME | AWS & Hashicorp Certified
https://www.linkedin.com/in/ernanicastro/

Follow

Responses (10)

Till Meineke

What are your thoughts?

Alexis Ware he
Feb 25

I explored your blog on ""A non-beginner Data Engineering Roadmap — 2025 Edition"", and I want to thank you for such an insightful breakdown of advanced data engineering concepts. Your discussion on modern OLAP, streaming, real-time processing, and... more

105 Reply

Ajay Malik
Feb 20

> I'll focus on established and mainstream tools, setting aside emerging favorites in the data community such as DuckDB, Polars, and uv.

This was such a well-crafted piece! Following you now—hope we can exchange thoughts and follow each other!

52 Reply

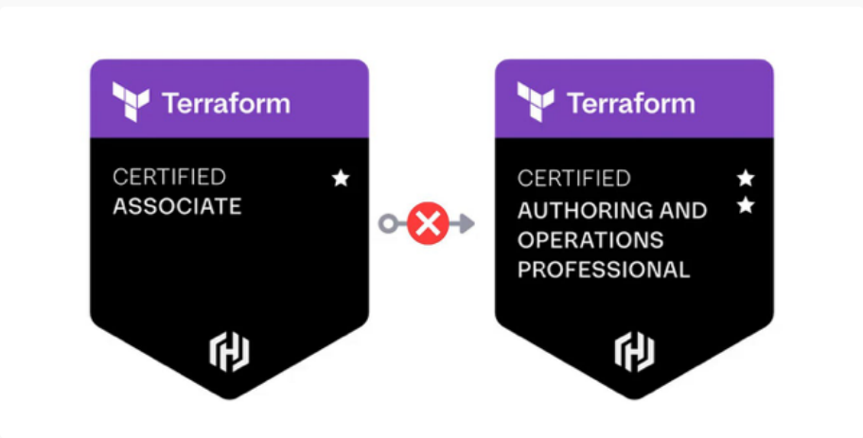
jadSen
Feb 19

A comprehensive out look of Data engineering/Services/Architect ... Well done!!

52 Reply

See all responses

More from Ernani Castro and Data Engineer Things

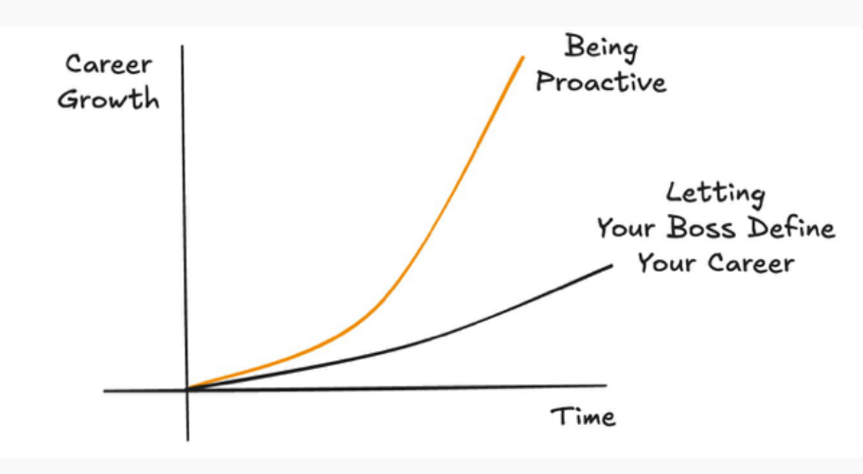

Terraform
CERTIFIED ASSOCIATE
Terraform
CERTIFIED AUTHORING AND OPERATIONS PROFESSIONAL

Ernani Castro

How I failed Terraform Authoring and Operations Pro with AWS...

This past week I had the opportunity to take the Terraform Pro and i failed it, here are my...

Oct 21, 2024 20

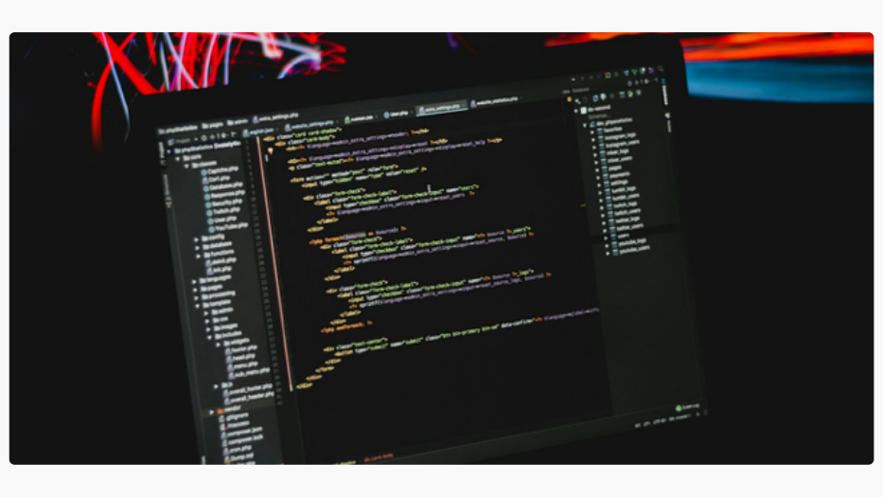


In Data Engineer Things by Ben Rogojan

5 Habits Of Highly Effective Data Engineers To Master in 2025

We are now nearly three months in to 2025. Likely whatever resolutions you've set for...

Feb 26 202 6

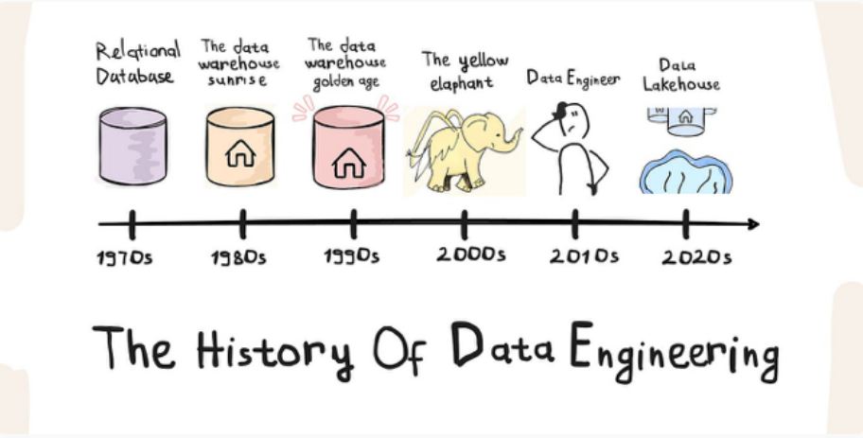


In Data Engineer Thin... by Kevin Meneses Gonzál...

5 Useful Iterator Functions in Python That Will Save You...

Has This Ever Happened to You?

Feb 11 132 1


Relational Database
The data warehouse sunrise
The data warehouse golden age
The yellow elephant
Data Engineer
Data Lakehouse
1970s
1980s
1990s
2000s
2010s
2020s
The History Of Data Engineering

In Data Engineer Things by Vu Trinh

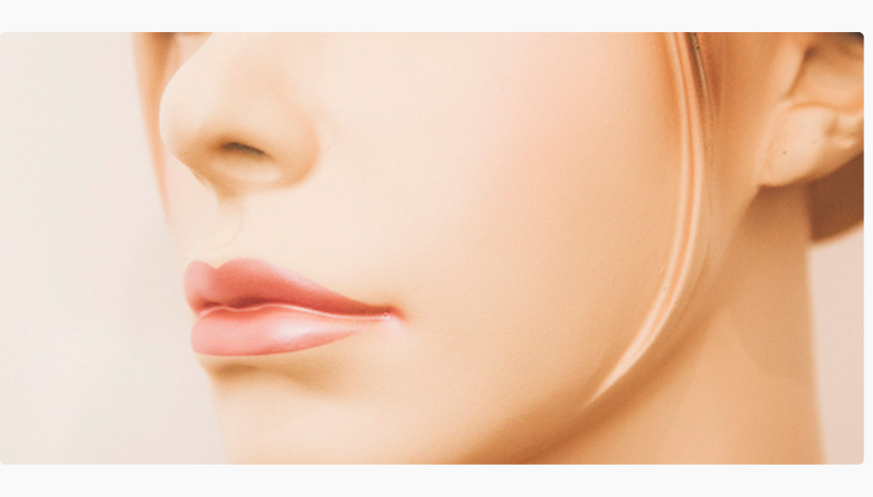
The History of Data Engineering

The most comprehensive one you've ever found on the internet

Jan 18 361 8

See all from Ernani Castro See all from Data Engineer Things

Recommended from Medium

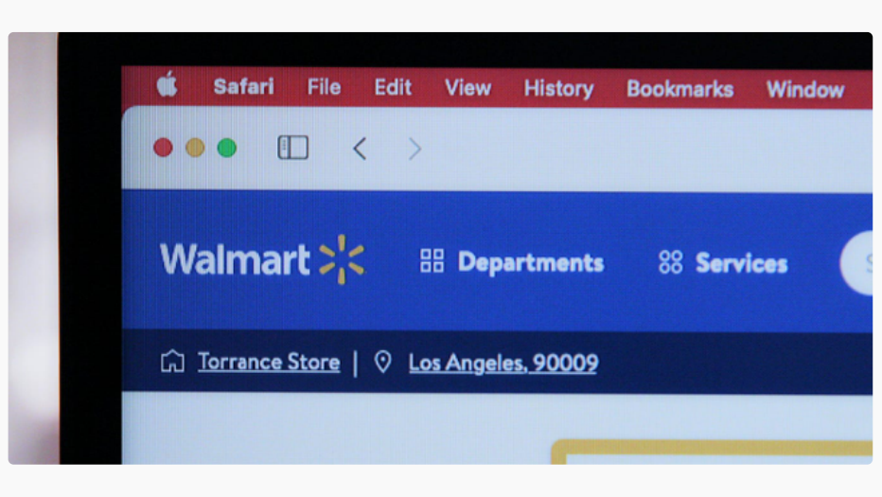


In Data Engineer Things by Shashwath Shenoy

Is Data Engineering Dying? The AI Takeover & The Future of Data...

For years, Data Engineers have been the Architects of scalable, robust Data pipelines...

Feb 17 67 5


Safari File Edit View History Bookmarks Window
Walmart
Departments
Services
Torrance Store | Los Angeles, 90009

Think Data

My Interview Experience for Senior Data Engineer at Walmart

I'm here to share my Senior Data Engineer interview experience with Walmart from...

Feb 22 160 5

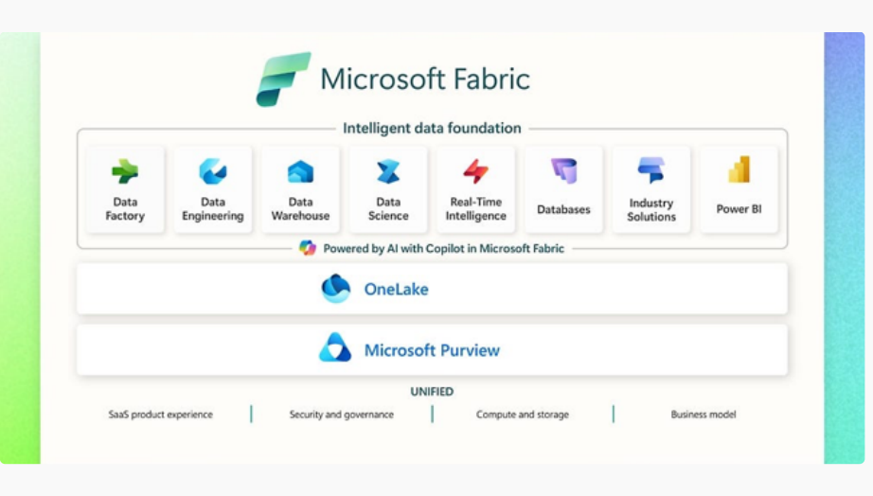
Lists

General Coding Knowledge
20 stories · 1933 saves

Stories to Help You Grow as a Software Developer
19 stories · 1616 saves

Leadership
62 stories · 523 saves

Good Product Thinking
13 stories · 828 saves

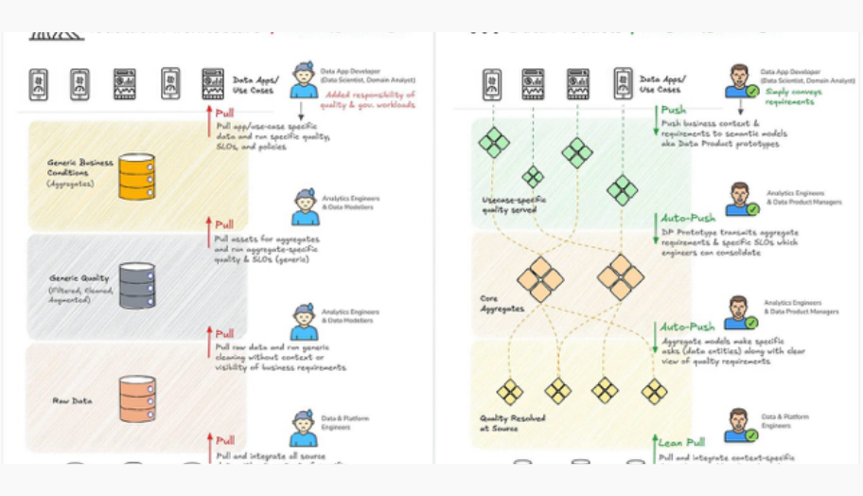

Microsoft Fabric
Intelligent data foundation
Data Factory
Data Engineering
Data Warehouse
Data Science
Real-Time Intelligence
Databases
Industry Solutions
Power BI
OneLake
Microsoft Purview

BastiaanRudolf

Microsoft Fabric: The Good, The Bad & The Ugly

A few weeks ago, I started a greenfield implementation of Fabric, Microsoft's next-...

Feb 20 84 8



Modern Data 101

Data Products: A Case Against Medallion Architecture

The Significance of Medallion, Crux of the Differences between the two 3-Tiered...

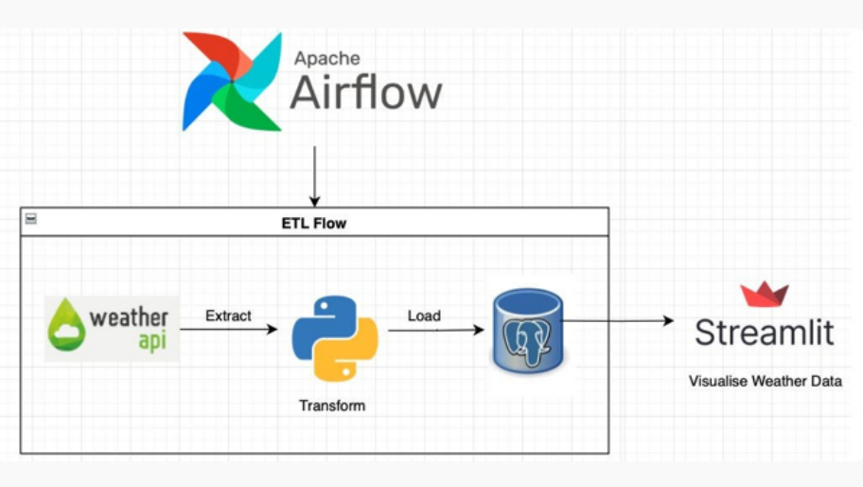
Feb 18 539 23

Mayurkumar Surani

Implementing End to end Change Data Capture (CDC) with PySpark...

Part 1: Foundation and Setup

Feb 21 89 6


Apache Airflow
ETL Flow
weather api
Extract
Transform
Load
Streamlit
Visualise Weather Data

In DataVidhya by Darshil Parmar

Understand Apache Airflow Like Never Before

In the world of data engineering, one of the most critical tasks you'll encounter is buildin...

Jan 29 154 8

See more recommendations